Parallel Programming Lab Report

Labs 1-2: Fork-Join Model, Thread Teams, and Numerical Integration

Student Name: Nursultan

Language / Environment: Python 3 (CPython, standard library only)

Section I: System and Hardware Specifications

Property	Value
CPU model	Apple M2
Physical cores	8
Logical cores	8 (no SMT / Hyper-Threading on Apple Silicon)
L1 data cache	64 KB per core
L2 cache	4 MB
Operating system	macOS (MacBook Air)
Runtime	Python 3 (CPython), standard library only (threading, multiprocessing)

Section II: Experimental Methodology

All timings were measured with Python's **time.perf_counter()**, a monotonic high-resolution timer. Because pure CPython was used (no JIT compiler such as Numba), no warm-up phase was required for Lab 1 and Lab 2 benchmarks.

Problem sizes (N) were reduced compared to the values suggested in the lab manual (originally 100,000,000) to keep run times practical on interpreted Python without a JIT compiler. The values actually used were: N = 2,000,000 for the race-condition test (Task 2.1), N = 1,000,000 for the critical-section test (Task 2.2, matching the manual exactly), and N = 5,000,000 for the strong-scaling test (Tasks 2.3-2.4). Task 1.1 was repeated 10 times as specified. Task 2.3 was repeated 5 times per thread count and averaged, as specified.

Architectural note: Lab 1 and Lab 2 Variants A/B use Python's **threading** module, which is required because a race condition or a lock-protected critical section both need threads that share the same memory space. Lab 2 Variant C (parallel reduction) instead uses **multiprocessing** (ProcessPoolExecutor), because CPython's Global Interpreter Lock (GIL) prevents threads from running Python bytecode on multiple cores simultaneously, so threading alone cannot show real speedup on a CPU-bound workload. Separate processes each have their own interpreter and GIL, so they can run truly in parallel across physical cores.

Section III: Empirical Results

Lab 1 - Task 1.1: Non-Determinism Verification

The 4-thread team was launched 10 consecutive times. In every run, the print order was identical: Master(0), Worker(3), Worker(1), Worker(2). This is explained by the artificial per-thread delay in the code, **time.sleep(0.001 * (thread_id % 3))**: threads 0 and 3 both sleep 0 ms, thread 1 sleeps 1 ms, and thread 2 sleeps 2 ms. Because these delays are much larger than normal OS scheduling jitter on an otherwise idle machine, the order was effectively repeatable in this environment, except for the tie between threads 0 and 3 (both 0 ms), which was resolved consistently in favor of thread 0 -- most likely because it was created and started first.

Lab 1 - Task 1.2: Thread Oversubscription Sweep

Threads (P)	Team create+join time (s)
1	0.000054
2	0.000076
4	0.000223
8	0.000355
16	0.000533
32	0.002214
64	0.002072

Time grows roughly monotonically with thread count, since this test measures only thread-creation and join overhead (empty worker function, no compute). The machine has 8 physical cores, so P = 16, 32 and 64 already oversubscribe the hardware by 2x, 4x and 8x respectively; the visible jump between P = 16 and P = 32 (0.53 ms to 2.2 ms) lines up with this transition.

Lab 1 - Task 1.3: CPU Core Saturation

4 threads, each computing 10,000,000 square roots, completed in **1.9198 s** of wall-clock time. A direct single-thread-vs-parallel comparison (Task 1.3b in the accompanying code, designed to numerically confirm the effect of the GIL) was implemented but not yet run; this remains a pending measurement.

Lab 2 - Task 2.1: Race Condition Quantification (N = 2,000,000)

Threads (P)	Computed Pi	Absolute Error	Time (s)
1	3.1415926536	2.01e-13	0.1742
2	2.4794955620	6.62e-01	0.2043
4	2.2777473540	8.64e-01	0.1749
8	2.1320400170	1.01e+00	0.1794

At P = 1 there is no race (a single thread updates the shared accumulator alone), and the result matches Pi almost exactly. From P = 2 onward, the unsynchronized **shared_sum[0] += term** update loses a large fraction of updates, and the error grows sharply with more threads (P = 8 loses more than 1.0 in absolute value out of Pi ~ 3.14, i.e. roughly a third of the true value).

Lab 2 - Task 2.2: Critical Section Overhead (N = 1,000,000)

Mode	Threads	Computed Pi	Time (s)
Serial baseline	1	3.1415926536	0.0898
Critical section	4	3.1415926536	0.1659

The critical section produces the correct result (matches serial Pi exactly), confirming mutual exclusion works, but at a cost: **84.9% overhead** versus the serial baseline, even though 4 threads were used. The lock must be acquired and released once per loop iteration, which fully serializes the accumulation step and adds synchronization overhead on top of the serial work itself.

Lab 2 - Tasks 2.3/2.4: Strong Scaling, Speedup, and Efficiency (N = 5,000,000)

Processes (P)	Avg. Time (s)	Speedup S(P)	Efficiency E(P)
1	0.4723	1.00x	1.00
2	0.2668	1.77x	0.88
4	0.1719	2.75x	0.69
8	0.2031	2.33x	0.29
16	0.3075	1.54x	0.10

Speedup increases up to P = 4 (2.75x), then **decreases** at P = 8 and P = 16, even though the machine has 8 physical cores. This indicates that process-creation and inter-process communication overhead (each ProcessPoolExecutor call spawns fresh worker processes and serializes task arguments and results) outweighs the benefit of additional parallelism once the workload per process becomes small relative to that fixed overhead. Efficiency falls from 1.00 at P = 1 to just 0.10 at P = 16, showing the scaling is far from linear beyond 4 processes on this workload size.

Section IV: Analytical Discussion (Draft)

The answers below are a starting draft grounded in the actual measured data above. They should be reviewed, verified, and rewritten in the student's own words before submission.

Lab 1

Question 1.1 - Non-Sequential Thread IDs

Thread print order is controlled by the operating system's scheduler, which decides when each runnable thread actually gets CPU time; this is independent of the order in which threads were created in the source code. In this experiment the order was in fact repeatable across all 10 runs (Master, Worker 3, Worker 1, Worker 2) because the code adds an artificial delay of 0, 1, or 2 milliseconds per thread, and this delay is much larger than typical OS scheduling jitter on an idle machine. Threads 0 and 3 share the same 0 ms delay and are the only pair whose relative order is not fully explained by the sleep values; that tie was broken the same way in every run, most likely due to creation order. On a busier machine, or without the sleep calls, true non-determinism from the scheduler would be much more visible.

Question 1.2 - Oversubscription Penalty

When the number of software threads P exceeds the number of physical cores (8 on this machine), the OS must time-slice multiple threads onto the same core, which requires context switching: saving one thread's registers, program counter, and stack pointer, and restoring another's. Each switch also risks evicting the previous thread's working set from the L1/L2 cache (cache thrashing), so the next thread that runs may have to reload data from a slower cache level or RAM. In the measured data, time grows from 54 microseconds at P = 1 to 2.2 milliseconds at P = 32 -- roughly a 40x increase for a 32x increase in thread count -- with a visible jump once P passes the physical core count of 8. Since this particular test measured only thread creation and join, not actual computation, it isolates OS/thread-management overhead rather than cache or compute contention.

Question 1.3 - Implicit Barrier

The implicit barrier at the end of a parallel region (here, calling join() on every worker thread) guarantees that the master thread does not continue executing until all worker threads have finished. Without this barrier, the master could read a shared result -- such as a partial sum -- before all threads finish writing to it, producing an incorrect or inconsistent value. It could also print output or write to a file that workers are still updating, or exit the program entirely and terminate still-running worker threads before they complete their work.

Question 1.4 - Hardware vs OS vs Language Threads

A hardware thread (SMT / Hyper-Threading) lets one physical CPU core execute two instruction streams by duplicating parts of the core's front end (registers, program counter) while sharing execution units; the Apple M2 in this machine does not implement SMT, which is why sysctl reports the same number (8) for both physical and logical cores. An OS kernel thread is a unit scheduled by the operating system's scheduler; it has its own stack and is what Python's threading module creates, but in CPython these threads are still serialized for Python bytecode execution by the GIL. A green/virtual thread (for example, Python's asyncio tasks or Java's virtual threads) is scheduled

entirely by the language runtime rather than the OS kernel, which makes context switching between them far cheaper, but it does not by itself provide CPU-bound parallelism across cores.

Lab 2

Question 2.1 - Memory Incoherency

The statement **shared_sum[0] += term** is not a single CPU instruction; it compiles down to a load of the current value, an add, and a store of the new value back to memory. If two threads interleave so that both load the same old value before either one stores its update, the second store overwrites the first, and one term is lost -- this is the classic lost-update problem. The measured data shows this clearly: absolute error grows from essentially zero at P = 1 (no race) to about 0.86 at P = 4 and just over 1.0 at P = 8, meaning a large fraction of the 2,000,000 terms were silently dropped as more threads increased the number of interleaving opportunities.

Question 2.2 - Reduction Trees vs Critical Sections

A critical section forces every single update through one lock, one at a time, so its cost scales as O(P) in contention regardless of how many cores are available -- the measured 84.9% overhead versus the serial baseline for just 4 threads on 1,000,000 steps shows this cost directly. A reduction tree instead lets each worker accumulate into a fully private accumulator with no synchronization at all, and only combines the P partial results at the very end, which needs on the order of log2(P) combination steps. The multiprocessing-based reduction in Task 2.3/2.4 is a one-level version of this pattern (each process computes an independent partial sum, combined once in the parent process), and it is the only variant in this lab that shows real speedup as P increases.

Question 2.3 - Amdahl's Law

If 5% of a program's execution time is strictly serial, Amdahl's Law gives a theoretical maximum speedup of 1 / 0.05 = 20x with infinitely many processors. The measured speedup in this lab flattens far below that limit -- it peaks at 2.75x at P = 4 and then actually **drops** to 2.33x at P = 8 and 1.54x at P = 16. This is not an Amdahl's Law ceiling; it is a case of secondary architectural factors dominating first: process creation and teardown overhead for every ProcessPoolExecutor call, serialization (pickling) of task arguments and results between processes, and oversubscription once P exceeds the machine's 8 physical cores.

Question 2.4 - Hardware Atomic Primitives

Modern multi-core CPUs implement atomic read-modify-write operations (such as java.util.concurrent.atomic types, or the LOCK-prefixed instructions like LOCK ADD or CMPXCHG on x86, and load-link/store-conditional pairs on ARM) by combining a cache coherence protocol -- typically MESI (Modified, Exclusive, Shared, Invalid) -- with a temporary exclusive lock on the relevant cache line. Before an atomic update, the owning core acquires the cache line in the Modified or Exclusive state, forcing other cores' copies to Invalid via bus snooping, performs the update, and only then allows the line to be shared again. This guarantees no other core can observe or modify that memory location mid-operation, at the cost of the cache-line invalidation traffic seen in false sharing scenarios (covered in Lab 4).

Section V: Conclusions

Across Labs 1 and 2, the central lesson was that Python's Global Interpreter Lock changes which concurrency mechanism is appropriate for which goal. Threading remains correct and useful for demonstrating race conditions and lock-based mutual exclusion, because both require shared memory, but it cannot deliver real CPU-bound speedup. Multiprocessing recovers real parallelism by using separate interpreters, but introduces its own overhead (process creation, serialization) that becomes the dominant cost once the number of processes exceeds the physical core count or the per-process workload becomes too small. Both labs also confirmed textbook concurrency hazards -- lost updates without synchronization, and high lock-contention overhead with synchronization -- with concrete measured numbers rather than just theory.